МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

ЗВІТ

з лабораторної роботи № 6
«Автоматизований контроль захищеності у конвеєрі CI/CD»
з дисципліни «Безпека програмного забезпечення»

Варіант 14

Виконав:
здобувач групи ІП-34
Скачков І. О.

Перевірив:
Курченко О. А.

Київ — 2026

1 МЕТА ТА ЗАВДАННЯ РОБОТИ

Мета роботи — навчитися вбудовувати автоматизовані перевірки захищеності в конвеєр CI/CD і налаштовувати критерій проходження (quality gate) — умову, за якою складання зупиняється в разі виявлення дефекту безпеки.

Завдання роботи:

1) налаштувати конвеєр зі стадіями статичного аналізу, аналізу складу ПЗ і сканування секретів;
2) розробити власне правило Semgrep, що виявляє характерний небезпечний шаблон коду;
3) виконати аналіз складу ПЗ і сформувати SBOM у форматі CycloneDX;
4) реалізувати в gate.py критерій проходження варіанта;
5) застосувати конвеєр до вразливого застосунку ЛР5 і довести, що він виявляє відомі вразливості;
6) оцінити частку хибних спрацювань і обґрунтувати налаштування критерію.

Параметри варіанта отримано командою `python -m common.variants 14 --lab 6`:

```
  Варіант 14

    LAB6
      sast: Semgrep + власне правило
      gate: блокувати за появи нових знахідок відносно базової лінії
```

Таблиця 1.1 – Індивідуальне завдання (варіант 14)

Параметр	Значення
Інструмент SAST	Semgrep + власне правило
Критерій проходження	блокувати за появи нових знахідок відносно базової лінії

2 ТЕОРЕТИЧНЕ ОБҐРУНТУВАННЯ ПРИЙНЯТИХ РІШЕНЬ

2.1 Критерій «нові знахідки відносно базової лінії»

Застосунок ЛР5 уже містить десятки знахідок SAST. Критерій «будь-яка знахідка HIGH» блокував би кожне складання й був би непридатний для такого

«спадкового» коду. Критерій базової лінії фіксує наявні знахідки як технічний борг окремим комітом (baseline.json) і блокує лише ті, яких там немає, тобто дефекти, внесені новою зміною. Формула (7.2) для варіанта: g = (N_нових > 0).

Ключове рішення — відбиток знахідки. Номер рядка в нього не входить: (інструмент, правило, файл, нормалізований текст рядка). Інакше будь-яка правка вище за кодом зсувала б рядки й робила старі знахідки «новими». Однакові знахідки рахуються кратно (multiset), тому друга копія вже відомої вади теж буде новою. Критерій діє за принципом fail closed: відсутній або пошкоджений звіт сканера, помилка виконання Semgrep чи відсутня базова лінія блокують складання, а не тлумачаться як «0 знахідок».

2.2 Власні правила Semgrep

Пробний прогін показав, що реєстр p/python знаходить SQL-ін'єкцію, MD5 та ін'єкцію команди, але не бачить трьох вразливостей мого варіанта ЛР5: відсутності перевірки прав (CWE-862) і обох XSS через фільтр |safe у шаблонах (CWE-79). Тому додано два власні правила. Головне — skachkov-flask-admin-route-without-authz. Відсутність перевірки прав є вадою логіки, а не викликом «небезпечної функції», тож загального шаблону для неї немає; правило кодує конвенцію проєкту: обробник маршруту /admin… має отримати current_user() і перевірити user["role"]. Допоміжне правило skachkov-jinja-safe-filter шукає |safe у шаблонах .html, яких p/python не аналізує.

2.3 Локальний прогін конвеєра

Workflow GitHub Actions виконується лише з кореня репозиторію, а ЛР6 лежить у підкаталозі монорепозиторію 7-sem. Тому конвеєр виконано локально: run_local.py запускає ті самі команди, що й security.yml, у Docker (образ інструментів із закріпленими версіями та офіційний образ gitleaks). Стани конвеєра зафіксовано кодами завершення, що методичка допускає як альтернативу знімкам Actions (крок 8). Файл security.yml оновлено й готовий до запуску в окремому репозиторії з коренем lab06_cicd.

3 ОПИС РОЗРОБЛЕНОГО РІШЕННЯ

3.1 Структура та програмні засоби

```
SoS/lab6/lab06_cicd/
  .github/workflows/security.yml   конвеєр: sast, sca, secrets
  rules/insecure_patterns.yml       правила Semgrep (+2 власні)
  gate.py                           критерій: нові знахідки vs baseline.json
  baseline.json                     базова лінія (окремий коміт)
  run_local.py                      локальний прогін конвеєра в Docker
  sca_cvss.py                       оцінки CVSS для звіту pip-audit (OSV)
  Dockerfile, requirements.txt      закріплені версії інструментів
  audit_target.txt                  ціль SCA (каркас курсу)
  target/                           копія застосунку ЛР5 — ціль SAST
```

Версії: Bandit 1.7.10, Semgrep 1.86.0, pip-audit 2.7.3, cyclonedx-bom 4.6.1 (як у requirements.txt каркаса), gitleaks 8.18.4, Python 3.12. У Dockerfile додано setuptools 69.5.1: без пакета pkg_resources Semgrep 1.86 у slim-образі Python 3.12 не запускається.

3.2 Схема конвеєра

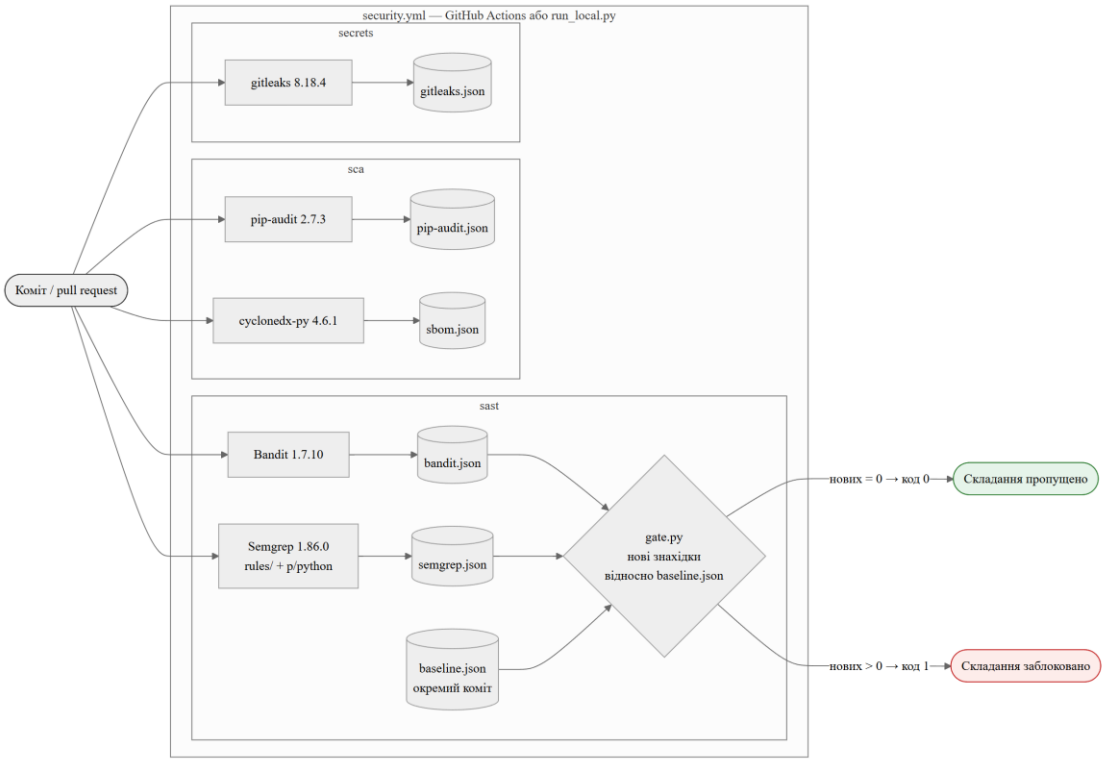


Рисунок 3.1 – Конвеєр контролю захищеності та критерій проходження

3.3 Власні правила Semgrep

```
# Головне правило: адміністративний маршрут Flask без перевірки ролі (CWE-862).
# Відсутність перевірки прав — вада логіки, а не «небезпечна функція», тому загальні
# правила її не знаходять. Правило кодує конвенцію проєкту: обробник маршруту /admin...
# має отримати current_user() і перевірити роль (user["role"]).
```

```
  - id: skachkov-flask-admin-route-without-authz
    languages: [python]
    severity: ERROR
    message: >-
      Адміністративний маршрут Flask без перевірки прав (CWE-862, OWASP A01:2021):
      обробник не перевіряє роль поточного користувача, тож маршрут доступний будь-кому.
      Отримайте user = current_user() і поверніть 302/403, якщо user["role"] != "admin".
    metadata:
      cwe: "CWE-862: Missing Authorization"
      owasp: "A01:2021 - Broken Access Control"
      author: "Скачков І. О., варіант 14"
    patterns:
      - pattern: |
          @app.route($PATH, ...)
          def $FUNC(...):
              ...
      - metavariable-regex:
          metavariable: $PATH
          regex: ^["']/admin
      - pattern-not: |
          @app.route($PATH, ...)
          def $FUNC(...):
              ...
              $USER = current_user()
              ...
              if <... $USER["role"] ...>:
                  ...
              ...
      # Знахідку прив'язано до імені обробника: відбиток не змінюється, коли редагують
тіло.
      - focus-metavariable: $FUNC


  # Допоміжне правило: фільтр |safe у шаблонах Jinja2 вимикає автоекранування (CWE-79).
  # Реєстр p/python шаблонів .html не аналізує, тому відображений і збережений XSS ЛР5
  # без цього правила лишаються невидимими.
  - id: skachkov-jinja-safe-filter
    languages: [regex]
    severity: WARNING
    message: >-
      Фільтр |safe вимикає автоекранування Jinja2 — вихід без екранування дає XSS
      (CWE-79, OWASP A03:2021). Приберіть |safe або екрануйте значення під контекст
виводу.
    metadata:
      cwe: "CWE-79: Improper Neutralization of Input During Web Page Generation"
      owasp: "A03:2021 - Injection"
      author: "Скачков І. О., варіант 14"
    paths:
      include: ["*.html"]
    pattern-regex: '\{\{[^}]*\|\s*safe\b[^}]*\}\}'
```

Шаблон головного правила: метазмінна $PATH маршруту має починатися з /admin (metavariable-regex), а тіло обробника не містить отримання current_user() із перевіркою $USER["role"] (pattern-not з оператором <... ...>). focus-metavariable прив’язує знахідку до імені функції, тож відбиток не змінюється, коли редагують тіло. Правило виявляє CWE-862 (OWASP A01).

3.4 Критерій проходження (gate.py)

Відбиток і пошук нових знахідок:

```
def fingerprint(f: dict) -> str:
    return f'{f["tool"]}|{f["rule"]}|{f["path"]}|{f["text"]}'
```

```
def new_findings(current: list[dict], baseline: Counter) -> list[dict]:
    """Знахідки, яких немає в базовій лінії (з урахуванням кратності)."""
    seen: Counter = Counter()
    new = []
    for f in current:
        fp = fingerprint(f)
        seen[fp] += 1
        if seen[fp] > baseline.get(fp, 0):
            new.append(f)
    return new
```

Рішення в main() після завантаження звітів і базової лінії:

```
    new = new_findings(current, baseline)
    gone = sum((baseline - Counter(fingerprint(f) for f in current)).values())
    if gone:
        print(f"Усунено знахідок відносно базової лінії: {gone} (базову лінію можна
оновити)")


    # Сумісність із каркасом: явно заданий MAX_HIGH додатково обмежує кількість HIGH.
    if "MAX_HIGH" in os.environ:
        high, threshold = bandit_high(load("bandit.json")), int(os.environ["MAX_HIGH"])
        if high > threshold:
            print(f"::error::Quality gate: {high} знахідок HIGH перевищує поріг
{threshold}")
            return 1


    if new:
        for f in new:
            print(f'::error file={f["path"]},line={f["line"]}::Нова знахідка
{f["tool"]}/{f["rule"]} '
                  f'[{f["severity"]}]: {f["text"][:90]}')
        print(f"::error::Quality gate: {len(new)} нових знахідок відносно базової лінії —
"
              "складання заблоковано")
        return 1
    print("Нових знахідок відносно базової лінії: 0")
    print("Quality gate: пройдено")
    return 0
```

Кожна нова знахідка друкується в форматі `::error file=…,line=…::`, який GitHub Actions показує анотацією в рядку коду. Ненульовий код завершення gate.py робить стадію sast червоною й зупиняє конвеєр. Перевірку HIGH каркаса збережено як додаткову: вона діє, лише якщо явно задано MAX_HIGH. Базову лінію записує `python gate.py --update-baseline`.

3.5 Стадія sast у security.yml

```
      - name: Bandit
        run: bandit -r target -f json -o bandit.json -q || true
      - name: Semgrep (власні правила + реєстр p/python)
        run: >-
          semgrep --metrics=off --disable-version-check
          --config rules/ --config p/python
          --json --output semgrep.json target || true
      # Варіант 14: нові знахідки відносно baseline.json → ненульовий код → стадія
червона.
      - name: Quality gate (нові знахідки відносно базової лінії)
```

```
run: python gate.py
```

4 РЕЗУЛЬТАТИ

4.1 Знахідки SAST на застосунку ЛР5

На цілі target/ (копія застосунку ЛР5 з 13 позначеними вразливостями) Semgrep дав 20 знахідок (ERROR — 10, WARNING — 10), Bandit — 12 (HIGH — 3, MEDIUM — 7, LOW — 2). Зведену таблицю з оцінкою кожної знахідки наведено в табл. 4.1; власні правила виділено.

Таблиця 4.1 – Зведена таблиця знахідок конвеєра на застосунку ЛР5

Інструмент	Правило	CWE	Маршрут / місце	К-сть	Оцінка
Semgrep	tainted-sql-string	CWE-89	/login (app.py:75)	1	істинне (ЛР5 №2)
Semgrep	insecure-hash-algorithm-md5, md5-used-as-password, python-md5-for-password	CWE-327/916	_weak_hash (app.py:101)	3	істинне (№4), дублікати
Semgrep	ті самі 3 правила MD5	CWE-327/916	seed.py:69	3	істинне (та сама вада №4)
Semgrep	**skachkov-flask-admin-route-without-authz**	CWE-862	/admin/users (app.py:214)	1	істинне (№12)
Semgrep	**skachkov-flask-admin-route-without-authz**	CWE-862	/admin/ping (app.py:235)	1	істинне (немає перевірки прав)
Semgrep	subprocess-injection, python-subprocess-shell-true, dangerous-subprocess-use, subprocess-shell-true	CWE-78	/admin/ping (app.py:238–239)	4	істинне (№13), дублікати
Semgrep	directly-returned-format-string, raw-html-format ×2	CWE-79	/admin/ping (app.py:241)	3	істинне (вивід ping із host без екранування)
Semgrep	avoid_app_run_with_bad_host	CWE-668	app.py:250, internal.py:31	2	**хибне** (0.0.0.0 у контейнері)
Semgrep	**skachkov-jinja-safe-filter**	CWE-79	/search (search.html:8)	1	істинне (№6)
Semgrep	**skachkov-jinja-safe-filter**	CWE-79	/document (document.html:9)	1	істинне (№7)
Bandit	B608	CWE-89	/login (app.py:76)	1	істинне (№2)
Bandit	B324 (HIGH)	CWE-327	app.py:101, seed.py:69	2	істинне (№4)
Bandit	B602 (HIGH)	CWE-78	/admin/ping (app.py:239)	1	істинне (№13)
Bandit	B301	CWE-502	/prefs (app.py:207)	1	істинне (№11)
Bandit	B310	CWE-22 за Bandit (фактично CWE-918)	/fetch (app.py:195)	1	істинне (№10, загальна перевірка)
Bandit	B403, B404 (LOW)	CWE-502, CWE-78	імпорт pickle, subprocess	2	**хибне** (сам імпорт не вада)
Bandit	B108	CWE-377	app.py:29, seed.py:85	2	**хибне** (БД у /tmp контейнера)
Bandit	B104	CWE-605	app.py:250, internal.py:31	2	**хибне** (як вище)

Відповідність знахідок тринадцяти відомим вразливостям застосунку ЛР5 — у табл. 4.2.

Таблиця 4.2 – Які вразливості ЛР5 виявив статичний аналіз

№	CWE	Вразливість	Semgrep	Bandit	Результат
1	CWE-798	жорстко закодований секрет	—	—	не виявлено
2	CWE-89	SQL-ін'єкція /login	tainted-sql-string	B608	виявлено
3	CWE-209	текст помилки СУБД	—	—	не виявлено
4	CWE-916	MD5 без солі	3 правила	B324	виявлено
5	CWE-639	IDOR /document	—	—	не виявлено
6	CWE-79	відображений XSS /search	власне правило	—	виявлено
7	CWE-79	збережений XSS /comment	власне правило	—	виявлено
8	CWE-352	CSRF /profile	—	—	не виявлено
9	CWE-22	обхід каталогів /download	—	—	не виявлено
10	CWE-918	SSRF /fetch	—	B310	лише Bandit
11	CWE-502	pickle /prefs	—	B301	лише Bandit
12	CWE-862	немає перевірки прав /admin/users	власне правило	—	виявлено
13	CWE-78	ін'єкція команди /admin/ping	4 правила	B602	виявлено

4.2 Власні правила

Головне правило дало 2 збіги на базовому стані (admin_users, admin_ping) і 1 новий у заблокованому стані (admin_export); допоміжне — 2 збіги (search.html, document.html). Хибних серед 4 базових збігів немає: /admin/ping так само не перевіряє прав. Негативна перевірка — прогін правил по виправленому коду ЛР5 (SoS/lab5): обробник admin_users, де після ЛР5 є перевірка ролі, і шаблон search.html без |safe правила вже не позначають, а невиправлені admin_ping і document.html позначають.

4.3 Аналіз складу ПЗ і SBOM

pip-audit знайшов у audit_target.txt 5 вразливостей у 2 пакетах. pip-audit 2.7.3 не подає оцінок CVSS, тому sca_cvss.py бере вектори CVSS v3 з бази OSV за GHSA-аліасами й обчислює базову оцінку за формулами CVSS v3.1:

```
pyyaml  5.3.1   CVE-2020-14343  CVSS 9.8 [CRITICAL]
CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H  fix 5.4
jinja2  2.11.3  CVE-2024-56326  CVSS 7.8 [MODERATE]
CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H  fix 3.1.5
jinja2  2.11.3  CVE-2024-22195  CVSS 5.4 [MODERATE]
CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N  fix 3.1.3
jinja2  2.11.3  CVE-2024-34064  CVSS 5.4 [MODERATE]
CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N  fix 3.1.4
jinja2  2.11.3  CVE-2025-27516  CVSS None [MODERATE]  лише v4:
CVSS:4.0/AV:L/AC:L/AT:P/PR:L/UI:P/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N  fix 3.1.6
Максимальна оцінка: 9.8; з CVSS ≥ 7.0: 2; з CVSS ≥ 4.0: 4 з 5
```

SBOM sbom.json — коректний документ CycloneDX 1.5 із 2 компонентами (Jinja2 2.11.3, PyYAML 5.3.1).

4.4 Сканування секретів

До каталогу додано синтетичний файл target/leaked_config.py із випадковим рядком у форматі токена GitHub (позначено «ТЕСТОВИЙ КЛЮЧ — не використовувати поза стендом», не комітився). gitleaks виявив його: RuleID github-pat, File target/leaked_config.py, StartLine 3, Match REDACTED (ключ --redact не записує секрет у звіт). Після видалення файлу повторний прогін дав порожній звіт; у робочому стані конвеєра й в історії комітів секретів немає.

4.5 Стани конвеєра

Стани зафіксовано окремими комітами; для кожного виконано повний прогін run_local.py (табл. 4.3, рис. 4.1–4.3).

Таблиця 4.3 – Стани конвеєра та коди завершення

Коміт	Зміна	Нових знахідок	gate.py	Конвеєр
4239a33	конвеєр без базової лінії	—	1 (fail closed)	заблоковано
f175b5b	baseline.json — 32 знахідки	0	0	пропущено
a41e308	новий маршрут /admin/export без перевірки прав	1	1	**заблоковано**
f69f2ad	перевірку ролі в /admin/export додано	0	0	**пропущено**

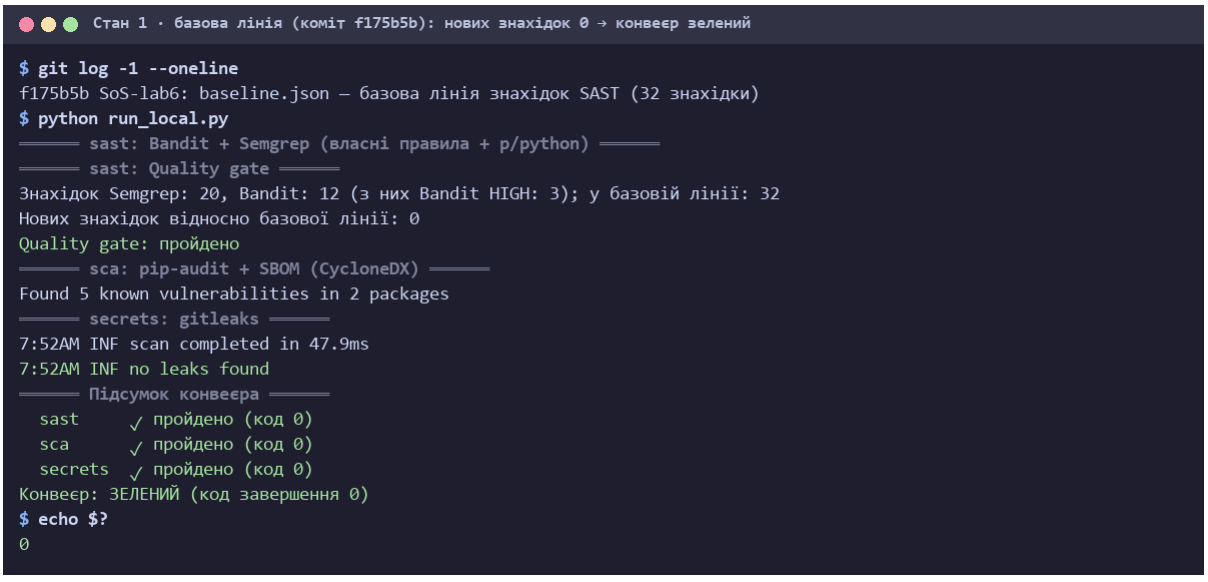


Рисунок 4.1 – Стан на базовій лінії (коміт f175b5b): код 0

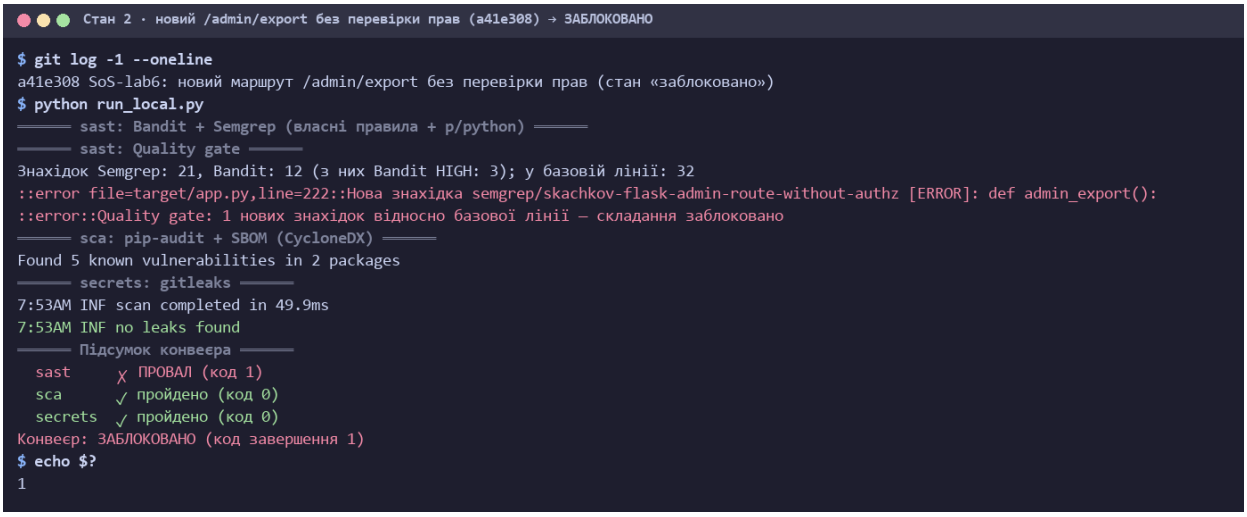


Рисунок 4.2 – Заблокований стан (коміт a41e308): код 1

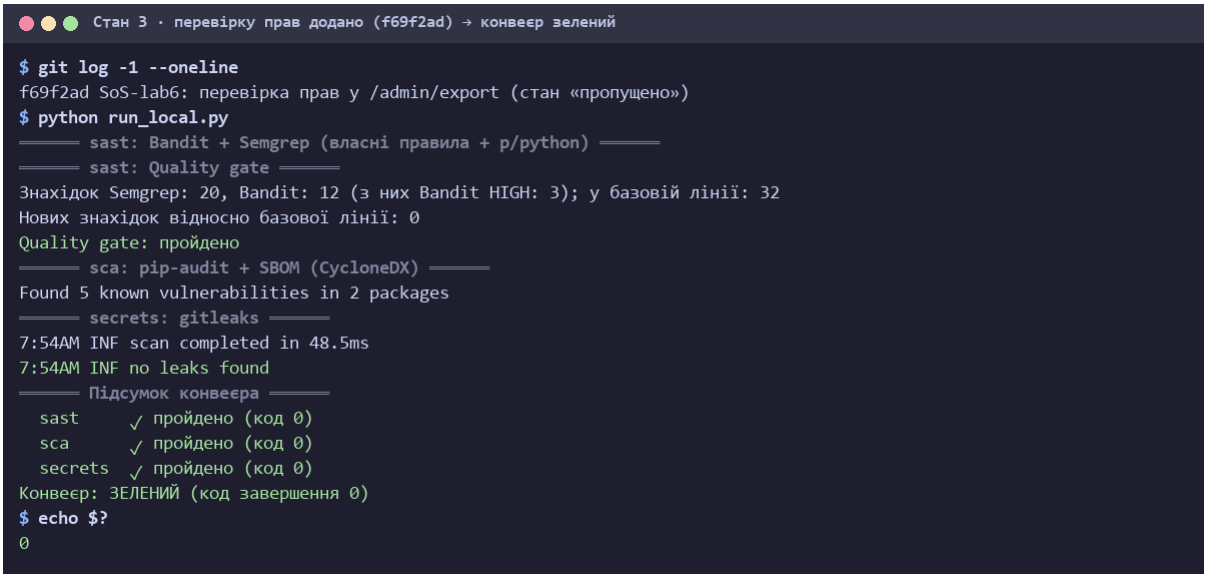


Рисунок 4.3 – Стан після усунення дефекту (коміт f69f2ad): код 0

5 АНАЛІЗ РЕЗУЛЬТАТІВ

5.1 Знахідки SAST і невидимі вразливості

Semgrep дав 20 знахідок: 10 ERROR і 10 WARNING; за CWE — CWE-78 (4), CWE-327/916 (6), CWE-79 (5), CWE-862 (2), CWE-89 (1), CWE-668 (2). Сам Semgrep покрив 6 із 13 відомих вразливостей (№2, 4, 6, 7, 12, 13), з них три (№6, 7, 12) — лише завдяки власним правилам; без них — 3 з 13. Разом із Bandit — 8 з 13 (додатково №10 і №11). Не виявлено №1 (секрет як значення за замовчуванням у os.environ.get), №3 (текст помилки в шаблоні), №5 (IDOR), №8 (CSRF) і №9 (обхід каталогів). IDOR і CSRF залежать від логіки: код без перевірки власника чи токена

синтаксично нічим не відрізняється від правильного, бо «правильність» визначає модель доступу застосунку, якої інструмент не знає. Власне правило CWE-862 спрацювало саме тому, що закодувало таку модель (конвенцію проєкту) явно.

5.2 Хибні спрацювання

Semgrep: 2 хибні з 20 (10 %); Bandit: 6 з 12 (50 %); разом 8 з 32 (25 %). Приклад — avoid_app_run_with_bad_host і B104 (прив'язка до 0.0.0.0). Усередині контейнера це потрібно, щоб застосунок був доступний через мережу Docker, а назовні порт публікується лише на 127.0.0.1 (docker-compose.yml), тож у цьому розгортанні вади немає. B403/B404 позначають сам імпорт pickle і subprocess, а B108 — шлях БД у /tmp одноразового контейнера. Окремо слід розрізняти дублікати: 4 знахідки на /admin/ping (рядки 238–239) — одна вада CWE-78, тому 20 знахідок Semgrep відповідають лише 9 різним дефектам.

5.3 Власні правила

Обидва правила спрацювали на реальному коді: головне — 2 збіги на базовому стані й 1 новий у заблокованому, допоміжне — 2 збіги; хибних немає, що підтверджено негативною перевіркою на виправленому коді ЛР5. Саме головне правило дало єдину нову знахідку, через яку критерій заблокував складання; реєстр p/python новий маршрут не помітив. Обмеження: правило розраховане на конвенцію «перевірка всередині обробника»; авторизація через декоратор (@admin_required) дала б хибне спрацювання, тому за такої конвенції правило доповнюють pattern-not-inside для декоратора.

5.4 SCA і поріг CVSS

pip-audit визнав вразливими обидві залежності. PyYAML 5.3.1 — CVE-2020-14343 (CVSS 9.8, виконання коду під час завантаження YAML). Jinja2 2.11.3 — CVE-2024-56326 (7.8), CVE-2024-22195 і CVE-2024-34064 (по 5.4), CVE-2025-27516 (у OSV лише вектор CVSS v4, рівень MODERATE). За порогом 7.0 блокували б 2 вразливості, за порогом 4.0 — 4 з 5: додаються дві середні. Коментар в audit_target.txt («у Jinja2 лише CVSS < 7.0») застарів: бази вразливостей

поповнюються, і закріплена залежність з часом стає ризикованішою. Тому SCA слід запускати на кожен коміт і за розкладом. CVE-2025-27516 без оцінки v3 показує ще один ризик: критерій лише за CVSS v3 її пропустить, тож відсутню оцінку треба тлумачити за принципом fail closed. Мій варіант не блокує за CVSS, тому стадія sca лише формує звіти.

5.5 Коли критерій блокує й пропускає

Критерій блокує, якщо серед знахідок Semgrep і Bandit є хоч одна, якої немає в базовій лінії (з урахуванням кратності), а також у разі збою сканера чи відсутності baseline.json. В інших випадках він пропускає. Коди завершення gate.py: 4239a33 — 1 (немає базової лінії), f175b5b — 0, a41e308 — 1 (нова знахідка admin_export), f69f2ad — 0. Зміна в a41e308 зсунула admin_ping на 10 рядків, але відбиток без номера рядка не дав хибної «нової» знахідки. Недолік критерію: наявні 32 знахідки не блокують складання. Тому базову лінію треба лише зменшувати (ratchet): gate.py друкує, скільки знахідок усунено, і пропонує оновити baseline.json.

5.6 Чому перевірки виконуються на кожен коміт

За формулою (7.1) C_n = C0 · 10^n вада, виявлена на етапі кодування (коміт), коштує в 10 разів менше, ніж на етапі тестування, і в 100 разів менше, ніж в експлуатації. Перевірка на кожен коміт знаходить дефект за хвилини після появи й прив’язує його до конкретної невеликої зміни — у стані a41e308 це один новий маршрут. Для критерію базової лінії це принципово: він вимірює приріст відносно попереднього стану, тому має сенс лише за безперервного запуску. Разовий аудит дав би лише загальний перелік без відповіді, хто й коли вніс ваду.

6 ВИСНОВКИ

1) Налаштовано конвеєр із трьома стадіями (sast, sca, secrets) і закріпленими версіями інструментів; локальний прогін run_local.py відтворює ті самі команди в Docker.
2) Розроблено два власні правила Semgrep: skachkov-flask-admin-route-without-authz (CWE-862) і skachkov-jinja-safe-filter (CWE-79). Вони додали

до покриття три вразливості ЛР5, невидимі для реєстру p/python, і не дали хибних спрацювань.

3) Реалізовано критерій «нові знахідки відносно базової лінії» з відбитком без номерів рядків і принципом fail closed. На базовій лінії (32 знахідки) конвеєр зелений; новий адмінмаршрут без перевірки прав дав 1 нову знахідку й код 1; після додавання перевірки ролі — код 0.
4) Статичний аналіз (Semgrep + Bandit) виявив 8 із 13 вразливостей ЛР5; IDOR, CSRF, обхід каталогів, розкриття помилки й секрет за замовчуванням лишилися невидимими. Частка хибних спрацювань: Semgrep — 10 %, Bandit — 50 %.
5) SCA визначив 5 вразливостей у 2 залежностях, найвища оцінка — CVSS 9.8 (PyYAML); SBOM CycloneDX 1.5 сформовано; gitleaks виявив синтетичний токен і дав порожній звіт після його видалення.

7 ВІДОМОСТІ ПРО ВИКОРИСТАННЯ ГЕНЕРАТИВНИХ ІНСТРУМЕНТІВ

Під час виконання роботи як допоміжний засіб використано генеративну модель Claude (Anthropic): для підготовки власних правил Semgrep, gate.py, run_local.py, sca_cvss.py, прогону конвеєра й оформлення звіту. Результати перевірено прогонами конвеєра в усіх станах; рішення здобувач перевірив і може пояснити під час захисту.

ПЕРЕЛІК ПОСИЛАНЬ

1) NIST SP 800-218. Secure Software Development Framework (SSDF) Version 1.1 / M. Souppaya, K. Scarfone, D. Dodson. – Gaithersburg : NIST, 2022.
2) Chess B., West J. Secure Programming with Static Analysis. – Boston : Addison-Wesley, 2007. – 624 p.
3) Semgrep Rule Syntax [Електронний ресурс]. – Режим доступу: https://semgrep.dev/docs/writing-rules/rule-syntax

4) CycloneDX Specification [Електронний ресурс] / OWASP Foundation. – Режим доступу: https://cyclonedx.org/
5) OSV – Open Source Vulnerabilities [Електронний ресурс]. – Режим доступу: https://osv.dev/
6) SLSA – Supply-chain Levels for Software Artifacts [Електронний ресурс] / OpenSSF. – Режим доступу: https://slsa.dev/
7) Kim G., Humble J., Debois P., Willis J. The DevOps Handbook. – 2nd ed. – Portland : IT Revolution Press, 2021. – 480 p.
8) Безпека програмного забезпечення. Частина 2. Лабораторний практикум / В. В. Соколовський. – Київ : КПІ ім. Ігоря Сікорського, 2026.

ДОДАТОК А

Репозиторій, коміти та артефакти

Репозиторій: https://github.com/ip34is/7-sem (папка SoS/lab6)

Коміт, що відповідає зданій роботі (стан «пропущено»): `f69f2ad003c021a7448ec07843d6792030735e9a`.

Коміт стану «заблоковано»: `a41e3085496fe95241ffcd31dcdb59c46401944f`; базова лінія: `f175b5bd9dcac93ad702d5b6d0f6f4da103af4d8`; налаштування конвеєра: `4239a33f42ea105c4220304d12fdf7b4d9f877fe`.

Стан роботи: https://github.com/ip34is/7-sem/tree/f69f2ad003c021a7448ec07843d6792030735e9a/SoS/lab6

Артефакти, що подаються разом зі звітом: bandit.json, semgrep.json, pip-audit.json, sbom.json, gitleaks.json, sca_cvss.json, gate.py, rules/insecure_patterns.yml, baseline.json. Звіти сканерів містять фрагменти коду вразливого застосунку, тому за правилами роботи до публічного репозиторію не додаються, а подаються лише разом зі звітом.

Відтворення (з папки SoS/lab6/lab06_cicd, потрібен Docker):

```
python run_local.py              # повний прогін; код завершення = результат
python run_local.py --baseline   # стадія sast + запис baseline.json
```

ДОДАТОК Б

Лістинг gate.py

```
"""Критерій проходження (quality gate) — ЛР6, варіант 14.

Критерій варіанта: блокувати складання за появи НОВИХ знахідок відносно базової лінії.

    g = (кількість нових знахідок SAST відносно baseline.json) > 0

Базова лінія (baseline.json) — зафіксований окремим комітом набір відбитків знахідок
Semgrep (власні правила + реєстр p/python) і Bandit. Знахідки, що вже є в базовій лінії
(технічний борг), складання не зупиняють; будь-яка знахідка, якої там немає, — зупиняє.

Відбиток знахідки не містить номера рядка: (інструмент, правило, файл, нормалізований
текст рядка знахідки). Тому зсув коду через сторонні правки не робить стару знахідку
«новою», а однакові знахідки рахуються кратно (multiset).

Запуск:
  python gate.py                     # рішення: код 0 — пройдено, 1 — заблоковано
  python gate.py --update-baseline   # перезаписати baseline.json поточними знахідками
Звіти bandit.json і semgrep.json мають лежати поряд (їх формує стадія sast).
"""
from __future__ import annotations

import json
import os
import re
import sys
from collections import Counter

BASELINE = "baseline.json"



def load(path: str) -> dict | None:
    """Читає JSON-звіт. Повертає None, якщо звіту немає або його пошкоджено."""
    try:
        with open(path, encoding="utf-8") as fh:
            return json.load(fh)
    except (OSError, json.JSONDecodeError):
        return None



def bandit_high(report: dict) -> int:
    return sum(1 for r in report.get("results", [])
               if r.get("issue_severity") == "HIGH")



def _norm(text: str) -> str:
    """Нормалізація тексту знахідки: без зайвих пробілів і переносів."""
    return re.sub(r"\s+", " ", text).strip()



def _source_line(path: str, line: int) -> str:
    try:
        with open(path, encoding="utf-8") as fh:
            return fh.read().splitlines()[line - 1]
    except (OSError, IndexError):
        return ""
```

```
def semgrep_findings(report: dict) -> list[dict]:
    out = []
    for r in report.get("results", []):
        rule = r["check_id"].split(".")[-1]
        out.append({"tool": "semgrep", "rule": rule, "path": r["path"],
                    "line": r["start"]["line"], "severity": r["extra"].get("severity", ""),
                    "text": _norm(r["extra"].get("lines", ""))})
    return out



def bandit_findings(report: dict) -> list[dict]:
    out = []
    for r in report.get("results", []):
        out.append({"tool": "bandit", "rule": r["test_id"], "path": r["filename"],
                    "line": r["line_number"], "severity": r.get("issue_severity", ""),
                    "text": _norm(_source_line(r["filename"], r["line_number"]))})
    return out



def fingerprint(f: dict) -> str:
    return f'{f["tool"]}|{f["rule"]}|{f["path"]}|{f["text"]}'



def new_findings(current: list[dict], baseline: Counter) -> list[dict]:
    """Знахідки, яких немає в базовій лінії (з урахуванням кратності)."""
    seen: Counter = Counter()
    new = []
    for f in current:
        fp = fingerprint(f)
        seen[fp] += 1
        if seen[fp] > baseline.get(fp, 0):
            new.append(f)
    return new



def collect() -> tuple[list[dict] | None, str]:
    """Знахідки обох інструментів або (None, причина) — якщо звіт відсутній чи збій
сканера."""
    semgrep, bandit = load("semgrep.json"), load("bandit.json")
    if semgrep is None or bandit is None:
        return None, "звіт semgrep.json або bandit.json відсутній чи пошкоджений"
    fatal = [e for e in semgrep.get("errors", []) if e.get("level") == "error"]
    if fatal:
        return None, f"Semgrep повідомив про {len(fatal)} помилок виконання (правила не
застосовано)"
    return semgrep_findings(semgrep) + bandit_findings(bandit), ""



def update_baseline() -> int:
    current, why = collect()
    if current is None:
        print(f"::error::Базову лінію не створено: {why}")
        return 1
    counts = Counter(fingerprint(f) for f in current)
    data = {"criterion": "new findings vs baseline (variant 14)",
            "total": len(current),
            "findings": [{"fingerprint": fp, "count": n} for fp, n in sorted(counts.items())]}
    with open(BASELINE, "w", encoding="utf-8") as fh:
        json.dump(data, fh, ensure_ascii=False, indent=2)
        fh.write("\n")
    print(f"Базову лінію записано: {len(current)} знахідок, {len(counts)} відбитків →
{BASELINE}")
    return 0



def main() -> int:
    current, why = collect()
```

```
    if current is None:
        # Fail closed: відсутній звіт — це збій сканера, а не «0 знахідок».
        print(f"::error::Quality gate: {why} — складання заблоковано")
        return 1
    base = load(BASELINE)
    if base is None:
        print(f"::error::Quality gate: {BASELINE} відсутній — створіть базову лінію "
              "окремим комітом (python gate.py --update-baseline)")
        return 1
    baseline = Counter({e["fingerprint"]: e["count"] for e in base["findings"]})


    semgrep_n = sum(1 for f in current if f["tool"] == "semgrep")
    print(f"Знахідок Semgrep: {semgrep_n}, Bandit: {len(current) - semgrep_n} "
          f"(з них Bandit HIGH: {bandit_high(load('bandit.json'))}); у базовій лінії:
{base['total']}")


    new = new_findings(current, baseline)
    gone = sum((baseline - Counter(fingerprint(f) for f in current)).values())
    if gone:
        print(f"Усунено знахідок відносно базової лінії: {gone} (базову лінію можна оновити)")


    # Сумісність із каркасом: явно заданий MAX_HIGH додатково обмежує кількість HIGH.
    if "MAX_HIGH" in os.environ:
        high, threshold = bandit_high(load("bandit.json")), int(os.environ["MAX_HIGH"])
        if high > threshold:
            print(f"::error::Quality gate: {high} знахідок HIGH перевищує поріг {threshold}")
            return 1


    if new:
        for f in new:
            print(f'::error file={f["path"]},line={f["line"]}::Нова знахідка
{f["tool"]}/{f["rule"]} '
                  f'[{f["severity"]}]: {f["text"][:90]}')
        print(f"::error::Quality gate: {len(new)} нових знахідок відносно базової лінії — "
              "складання заблоковано")
        return 1
    print("Нових знахідок відносно базової лінії: 0")
    print("Quality gate: пройдено")
    return 0



if __name__ == "__main__":
    sys.stdout.reconfigure(encoding="utf-8")
    sys.exit(update_baseline() if "--update-baseline" in sys.argv else main())
```

ДОДАТОК В

Лістинги security.yml і run_local.py

```
# .github/workflows/security.yml
# Конвеєр контролю захищеності для лабораторної роботи 6 (варіант 14).
# SAST: Semgrep + власні правила (rules/) і Bandit; критерій проходження —
# блокувати складання за появи НОВИХ знахідок відносно базової лінії baseline.json.
# Локально ті самі команди виконує run_local.py (Docker).
name: security


on:
  push:
  pull_request:
  workflow_dispatch:


permissions:
  contents: read   # мінімальні права токена (лекція 15)


jobs:
  sast:
    name: Статичний аналіз (SAST) + quality gate
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: Встановлення інструментів (закріплені версії)
        run: pip install -r requirements.txt
      # Ціль SAST — каталог target/ із копією застосунку ЛР5.
      # `|| true`: сканер лише формує звіт; рішення ухвалює gate.py (fail closed).
      - name: Bandit
        run: bandit -r target -f json -o bandit.json -q || true
      - name: Semgrep (власні правила + реєстр p/python)
        run: >-
          semgrep --metrics=off --disable-version-check
          --config rules/ --config p/python
          --json --output semgrep.json target || true
      # Варіант 14: нові знахідки відносно baseline.json → ненульовий код → стадія червона.
      - name: Quality gate (нові знахідки відносно базової лінії)
```

```
      run: python gate.py
    - name: Звіти SAST
      if: always()
      uses: actions/upload-artifact@v4
      with:
        name: sast-reports
        path: |
          bandit.json
          semgrep.json


sca:
  name: Аналіз складу ПЗ (SCA) та SBOM
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with:
        python-version: "3.12"
    - name: Встановлення інструментів (закріплені версії)
      run: pip install -r requirements.txt
    # Ціль SCA — навчальний перелік залежностей audit_target.txt.
    - name: Пошук уразливих залежностей
      run: |
        pip-audit -r audit_target.txt || true
        pip-audit -r audit_target.txt -f json -o pip-audit.json || true
    - name: Формування SBOM (CycloneDX)
      run: cyclonedx-py requirements audit_target.txt -o sbom.json || true
    - name: Звіти SCA
      if: always()
      uses: actions/upload-artifact@v4
      with:
        name: sca-reports
        path: |
          pip-audit.json
          sbom.json


secrets:
  name: Сканування секретів
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
      with:
        fetch-depth: 0   # уся історія: секрет у будь-якому коміті вважається скомпрометованим
    - name: gitleaks
      uses: gitleaks/gitleaks-action@v2
      env:
```

```
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

```
# run_local.py
"""Локальний прогін конвеєра .github/workflows/security.yml (ЛР6, варіант 14).


Повторює три задачі конвеєра тими самими командами, що й GitHub Actions, але в Docker
(образ інструментів із Dockerfile та офіційний образ gitleaks):
  sast    — Bandit, Semgrep (rules/ + p/python), quality gate (python gate.py);
  sca     — pip-audit (текст + JSON), SBOM CycloneDX;
  secrets — gitleaks (сканування робочого каталогу).


Запуск з каталогу lab06_cicd:
  python run_local.py              # повний прогін; код завершення = результат конвеєра
  python run_local.py --baseline   # стадія sast + запис baseline.json (окремий коміт)
"""
from __future__ import annotations


import os
import shutil
import subprocess
import sys
from pathlib import Path


HERE = Path(__file__).resolve().parent
IMAGE = "bpz-lab6-tools"
GITLEAKS = "zricethezav/gitleaks:v8.18.4"
DOCKER_FALLBACK = Path(os.environ.get("LOCALAPPDATA", "")) / "Programs/DockerDesktop/resources/bin/docker.exe"


# Ті самі команди, що в security.yml.
SAST = [
    "bandit -r target -f json -o bandit.json -q || true",
    "semgrep --metrics=off --disable-version-check --config rules/ --config p/python "
    "--json --output semgrep.json target >/dev/null 2>&1 || true",
]
GATE = "python gate.py"
SCA = [
    "pip-audit -r audit_target.txt || true",
    "pip-audit -r audit_target.txt -f json -o pip-audit.json >/dev/null 2>&1 || true",
    "cyclonedx-py requirements audit_target.txt -o sbom.json || true",
]



def docker() -> str:
```

```
    found = shutil.which("docker") or (str(DOCKER_FALLBACK) if DOCKER_FALLBACK.exists() else None)
    if not found:
        sys.exit("Docker не знайдено: встановіть і запустіть Docker Desktop")
    return found



def run(args: list[str]) -> int:
    return subprocess.run(args).returncode



def in_tools(cmd: str) -> int:
    """Команда в образі інструментів; каталог lab06_cicd змонтовано як /work."""
    return run([docker(), "run", "--rm", "-v", f"{HERE}:/work", "-w", "/work", IMAGE, "sh", "-c", cmd])



def ensure_image() -> None:
    probe = subprocess.run([docker(), "image", "inspect", IMAGE], capture_output=True)
    if probe.returncode != 0:
        print(f"Збираю образ {IMAGE} …", flush=True)
        if run([docker(), "build", "-q", "-t", IMAGE, str(HERE)]) != 0:
            sys.exit("Не вдалося зібрати образ інструментів")



def stage(title: str) -> None:
    print(f"\n═══ {title} ═══", flush=True)



def job_sast(baseline: bool = False) -> int:
    stage("sast: Bandit + Semgrep (власні правила + p/python)")
    for cmd in SAST:
        in_tools(cmd)
    stage("sast: Quality gate" + (" — запис базової лінії" if baseline else ""))
    return in_tools(GATE + (" --update-baseline" if baseline else ""))



def job_sca() -> int:
    stage("sca: pip-audit + SBOM (CycloneDX)")
    for cmd in SCA:
        in_tools(cmd)
    return 0                    # як у security.yml: стадія лише формує звіти (|| true)
```

```
def job_secrets() -> int:
    stage("secrets: gitleaks")
    return run([docker(), "run", "--rm", "-v", f"{HERE}:/repo", GITLEAKS, "detect",
                "--source", "/repo", "--no-git", "--redact", "--no-banner",
                "--report-format", "json", "--report-path", "/repo/gitleaks.json"])



def main() -> int:
    sys.stdout.reconfigure(encoding="utf-8")
    ensure_image()
    if "--baseline" in sys.argv:
        return job_sast(baseline=True)
    results = {"sast": job_sast(), "sca": job_sca(), "secrets": job_secrets()}
    stage("Підсумок конвеєра")
    for job, code in results.items():
        print(f"  {job:8} {'✓ пройдено' if code == 0 else '✗ ПРОВАЛ'} (код {code})")
    failed = any(results.values())
    print(f"Конвеєр: {'ЗАБЛОКОВАНО' if failed else 'ЗЕЛЕНИЙ'} (код завершення {int(failed)})")
    return int(failed)



if __name__ == "__main__":
    sys.exit(main())
```